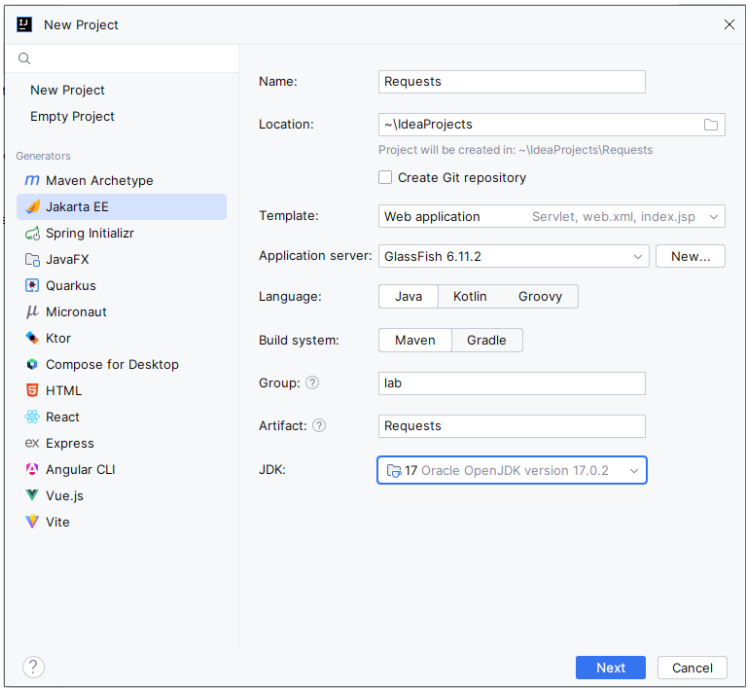
Jakarta EE: JSF + CDI + JPA + BV

Ćwiczenie 1

Celem ćwiczenia jest utworzenie prostej aplikacji bazodanowej umożliwiającej przeglądanie i dodawanie żądań serwisowych. Ćwiczenie pokazuje współpracę technologii JSF, CDI i JPA w ramach platformy Jakarta EE. Do realizacji ćwiczenia wymagane jest środowisko IntelliJ IDEA Ultimate wraz z serwerem aplikacji Payara i serwerem bazy danych H2 (dostępnym razem z serwerem aplikacji Payara). Oprócz podstawowych wtyczek obsługujących platformę Jakarta w IntelliJ IDEA Ultimate, wykorzystana będzie wtyczka „JPA Buddy". Jeśli nie jest jeszcze zainstalowana, to ją doinstaluj.

1. Uruchom IntelliJ IDEA i utwórz nowy projekt opcją New Project. W kreatorze projektu jako nazwę projektu podaj „Requests", zaznacz w panelu generatorów Jakarta EE, a następnie z listy szablonów wybierz Web Application. Upewnij się, że jako serwer aplikacji wybrany jest skonfigurowany wcześniej serwer Payara (widoczny jako GlassFish), Java jako język programowania, a Maven jako system budowania aplikacji. W polu Group wpisz „lab", a w polu Artifact pozostaw treść zaproponowaną przez kreator, zgodną z nazwą projektu. Jako wersję JDK wybierz 21. Kliknij przycisk Next.



   W drugim kroku kreatora projektu wybierz wersję platformy Jakarta Jakarta EE 11, a w panelu zależności zaznacz Web Profile. Kliknij przycisk Create.

   Obejrzyj strukturę projektu i przejrzyj zawartość pliku `pom.xml`.

2. Usuń z projektu utworzony przez kreator serwlet i stronę JSP.

3. Utwórz w projekcie jednostkę trwałości, w ramach której obiekty aplikacji będą zachowywane w bazie danych. W tym celu:

   a) Odszukaj w strukturze projektu plik `persistence.xml` (powinien zostać utworzony przez kreator projektu i znajdować się w podkatalogu META-INF) i otwórz go do edycji.

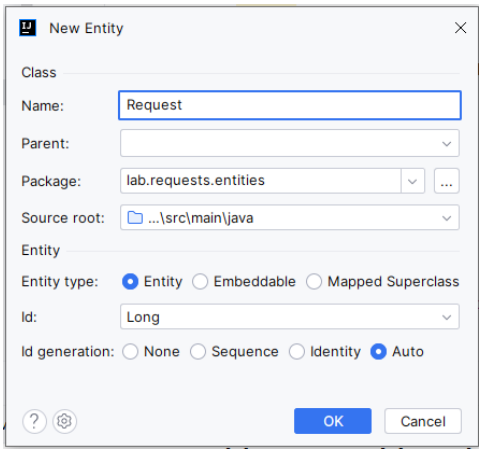
   b) Zastąp w kodzie źródłowym pliku `persistence.xml` cały element <persistence-unit> poniższą wersją, specyfikującą jednostkę trwałości powiązaną ze źródłem danych na serwerze aplikacji i wykorzystującą transakcje w standardzie JTA.

```
<persistence-unit name="RequestsPU" transaction-type="JTA">
  <jta-data-source>jdbc/__default</jta-data-source>
  <exclude-unlisted-classes>false</exclude-unlisted-classes>
  <properties>
    <property
     name="jakarta.persistence.schema-generation.database.action"
     value="create"/>
  </properties>
</persistence-unit>
```

4. Utworzenie klasy encji `Request` do reprezentowania żądań.

   a) Kliknij prawym przyciskiem myszy na węźle `src/main/java` i z menu kontekstowego wybierz New → JPA Entity.

   b) Jako nazwę klasy podaj `Request`, a jako nazwę pakietu `lab.requests.entities`. Wybierz z listy typ `Long` jako typ identyfikatora encji i zaznacz Auto jako sposób generowania wartości identyfikatorów (strategia dobrana do możliwości wykorzystywanego serwera bazy danych).



   c) Kliknij OK. Otwórz klasę encji do edycji i obejrzyj wygenerowany kod.

   d) Dodaj w klasie encji dwa prywatne pola: `requestDate` typu `LocalDate` i `requestText` typu `String`. Dodaj import klasy `java.time.LocalDate`. Wygeneruj publiczne gettery i settery dla dodanych pól (skorzystaj z kreatora Refactor → Encapsulate Fields).

5. Utwórz w projekcie klasę `lab.requests.data.EntityManagerProducer`, która będzie implementować bean CDI pozwalający uzyskać obiekt `EntityManager` w sposób pozwalający na jego wstrzyknięcie jak beana CDI z jednoczesnym zapewnieniem, że nie będzie on współdzielony przez różne żądania. W tym celu:

a) Z menu kontekstowego węzła `src/main/java` uruchom kreator Java Class.

b) Wprowadź nazwę klasy poprzedzoną nazwą pakietu.

c) Oznacz klasę adnotacją CDI `@Dependent`.

Komentarz: Zasięg `@Dependent` jest domyślny. Ustawiliśmy go jednak jawnie adnotacją, aby klasa producenta była uznana za bean CDI niezależnie od ustawienia atrybutu `bean-discovery-mode` w pliku `beans.xml`. Domyślne w Jakarta EE 10 ustawienie to „annotated". Aby klasa POJO nieoznaczona żadną adnotacją CDI była uznana za bean CDI, należałoby jawnie ustawić wartość wspomnianego atrybutu na „all".

d) Jako ciało klasy wklej poniższy kod. Zwróć uwagę na zwykłe wstrzyknięcie obiektu `EntityManager` i na metodę producenta, która „opakowuje" go jako bean CDI o zasięgu żądania.

```
    @PersistenceContext
    private EntityManager em;

    @Produces
    @RequestScoped
    public EntityManager getEntityManager() {
        return em;
    }
```

e) Wstaw poniższe instrukcje `import`. (Uwaga: W tym wypadku należy zwrócić uwagę by zaimportować adnotacje z właściwych pakietów, gdyż niektóre z nich powtarzają się w różnych pakietach.)

```
import jakarta.enterprise.context.RequestScoped;
import jakarta.enterprise.inject.Produces;
import jakarta.persistence.EntityManager;
import jakarta.persistence.PersistenceContext;
```

6. Utwórz w projekcie klasę `lab.requests.data.RequestRepository`, która będzie oferować funkcjonalność CRUD do obsługi encji `Request`. W tym celu:

a) Z menu kontekstowego węzła `src/main/java` uruchom kreator Java Class.

b) Wprowadź nazwę klasy poprzedzoną nazwą pakietu.

c) Oznacz klasę adnotacją `@ApplicationScoped`.

d) W ciele klasy umieść poniższą kod wstrzykujący obiekt `EntityManager` jako bean CDI.

```
    @Inject
    private EntityManager em;
```

Komentarz: Zalecany zasięg CDI dla beana implementującego funkcjonalność repozytorium to zasięg aplikacyjny, który sprawi że jedna instancja beana będzie współdzielona przez wszystkie żądania i sesje użytkowników. W przypadku gdyby do takiego beana `EntityManager` był wstrzyknięty zwyczajnie adnotacją `@PersistenceContext`, to z racji zasięgu aplikacyjnego beana repozytorium, byłby on również współdzielony przez wszystkie żądania, co stanowiłoby duży błąd, gdyż obiekty `EntityManager` nie są „thread-safe". W przyjętym rozwiązaniu

`EntityManager` jest „produkowany” z zasięgiem pojedynczego żądania przez metodę producenta zawartą w beanie o domyślnym zasięgu `@Dependent`, co jest odpowiednie w tym wypadku. CDI obsługuje wstrzykiwanie beana o mniejszym zasięgu do beana o większym zasięgu (co ma miejsce w naszym rozwiązaniu) poprzez obiekty proxy.

e) Umieść w ciele klasy poniżej wstrzyknięcia obiektu `EntityManager` poniższe metody repozytorium.

```
public void create(Request entity) {
    em.persist(entity);
}

public void edit(Request entity) {
    em.merge(entity);
}

public void remove(Request entity) {
    em.remove(em.merge(entity));
}

public Request find(Object id) {
    return em.find(Request.class, id);
}

public List<Request> findAll() {
    CriteriaQuery cq = em.getCriteriaBuilder().createQuery();
    cq.select(cq.from(Request.class));
    return em.createQuery(cq).getResultList();
}
```

f) Uzupełnij brakujące importy. Zwróć uwagę aby klasy/interfejsy JPA importować z pakietów o nazwach rozpoczynających się od „jakarta”, a klasę `Request` jako naszą własną klasę encji.

7. Ponieważ planujemy oprzeć interfejs użytkownika na technologii JSF, konieczne jest skonfigurowanie serwletu w roli Front Controller dla stron, które będą pełnić rolę widoków aplikacji. Otwórz do edycji plik `web.xml`. W obrębie elementu `<web-app>` umieść poniższe elementy.

```
<servlet>
    <servlet-name>Faces Servlet</servlet-name>
    <servlet-class>
       jakarta.faces.webapp.FacesServlet
    </servlet-class>
    <load-on-startup>1</load-on-startup>
</servlet>
<servlet-mapping>
    <servlet-name>Faces Servlet</servlet-name>
    <url-pattern>/faces/*</url-pattern>
</servlet-mapping>
```

8. Utwórz w projekcie nową stronę JSF (New → JSF/Facelets z menu kontekstowego węzła `webapp`). Jako jej nazwę podaj `requestsList`. Zwróć uwagę aby z dwóch dostępnych opcji tworzonego pliku wybrana była opcja JSF File. Modyfikacją zawartości strony zajmiemy się później.

9. Utwórz komponent CDI, który będzie pełnił rolę backing bean dla utworzonej przed chwilą strony JSF. W tym celu:

   a) Utwórz w projekcie klasę `lab.requests.backing.RequestsList`.

   b) Oznacz klasę adnotacją ustawiającą typowy dla komponentów backing bean zasięg (`@RequestScoped`).

   c) Oznacz klasę adnotacją @Named, dzięki której komponent CDI będzie dostępny z poziomu języka wyrażeń na stronie JSF. Nie musimy w adnotacji jawnie podawać nazwy, gdyż satysfakcjonuje nas domyślna zgodna z konwencją, czyli nazwa klasy z pierwszą literą zmienioną na małą (tj. „requestsList").

   d) Wstrzyknij referencję do komponentu repozytorium wstawiając w klasie poniższy kod (pamiętaj o niezbędnych importach!).

```
    @Inject
    private RequestRepository requestRepository;
```

   e) Poniżej wklej następującą metodę, która ma pośredniczyć w dostępie do metody komponentu repozytorium i udostępniać listę wszystkich obiektów encji w formie właściwości komponentu zarządzanego. Samodzielnie uzupełnij kod metody i zaimportuj wykorzystywane klasy (java.util.List oraz naszą klasę encji Request). Zapisz wszystkie zmiany.

```
public List<Request> getAllRequests() {
      ...
}
```

10. Przejdź do edycji źródła strony JSF (`requestsList.xhtml`). Zastąp opcjonalny element `<f:view>` (wraz z jego zawartością) poniższym kodem.

```
<h:head>
    <title>Facelet Title</title>
</h:head>
<h:body>
    <h:form>
        <h:dataTable value="#{requestsList.allRequests}"
                     var="item" border="1" >
            <h:column>
                <f:facet name="header">
                    <h:outputText value="Id"/>
                </f:facet>
                <h:outputText value="#{item.id}"/>
            </h:column>
            <h:column>
                <f:facet name="header">
                    <h:outputText value="RequestDate"/>
                </f:facet>
                <h:outputText value="#{item.requestDate}"/>
            </h:column>
            <h:column>
                <f:facet name="header">
                    <h:outputText value="RequestText"/>
```

```
                    </f:facet>
                    <h:outputText value="#{item.requestText}"/>
                </h:column>
            </h:dataTable>
        </h:form>
</h:body>
```

11. Usuń nieużywaną deklarację przestrzeni nazw, którą dodał kreator strony (tę powiązaną z prefiksem `ui`).

12. Otwórz do edycji plik `web.xml`. W obrębie elementu `<web-app>` umieść poniższy element ustawiający domyślą stronę startową aplikacji.

```
    <welcome-file-list>
        <welcome-file>faces/requestsList.xhtml</welcome-file>
    </welcome-file-list>
```

13. Zapisz wszystkie zmiany i uruchom aplikację. W przeglądarce powinna wyświetlić się strona z tabelką z listą żądań. (Na razie bez żadnych danych.)

14. Przejdź do edycji źródła utworzonej strony JSF i wstaw wewnątrz elementu `<h:form>` przed elementem `<h:dataTable>` poniższe trzy elementy (korzystając z podpowiedzi i autouzupełniania edytora tekstowego):

    - element `<h:outputText>` z atrybutem `value` o wartości „New request”
    - element `<h:inputText>` z atrybutem `id` o wartości „newReqInputText”
    - element `<h:commandButton>` z atrybutem `value` o wartości „Add”

15. Przejdź do edycji kodu klasy komponentu backing bean (`RequestsList`). Dodaj prywatną właściwość `newRequest` typu `String` z publicznymi metodami getter i setter.

16. Analogicznie do poprzedniego punktu ćwiczenia dodaj w klasie backing bean właściwość `requestsDataTable` typu `jakarta.faces.component.html.HtmlDataTable`.

17. Przejdź do edycji strony JSF. Dokonaj powiązania jej komponentów z komponentem backing bean dodając następujące atrybuty znaczników:

    - w elemencie `<h:inputText>`:
      `value="#{requestsList.newRequest}"`
    - w elemencie `<h:commandButton>`:
      `action="#{requestsList.addRequest}"`
    - w elemencie `<h:dataTable>`:
      `binding="#{requestsList.requestsDataTable}"`

18. Dodaj w klasie komponentu backing bean metodę, do której odnosi się przycisk na stronie JSF. Uzupełnij kod metody o operacje:

    - utworzenia instancji klasy `Request`
    - ustawienia w nowo utworzonym obiekcie daty żądania na bieżącą i tekstu żądania na wprowadzony do formularza
    - utrwalenia obiektu poprzez repozytorium

```
public String addRequest()
{
    ...
    setNewRequest("");
    return null;
}
```

19. Zapisz wszystkie zmiany. Uruchom aplikację. Przetestuj czy działa dodawanie nowych żądań serwisowych.

20. Przy próbie dodania nowego żądania powinien pojawić się wyjątek informujący o tym, że operacje modyfikujące bazę danych powinny być realizowane w transakcji. W przeciwieństwie do metod beanów starszej technologii EJB, metody beanów CDI nie są domyślnie transakcyjne. Oznacz więc metodę dodającą nowe żądanie w klasie komponentu backing bean (nie w klasie repozytorium!) adnotacją `@Transactional`. Dodaj stosowny import.

21. Dodaj w klasie komponentu backing bean metodę do usuwania obiektu encji `Request` reprezentowanego przez bieżący wiersz w komponencie `Data Table`. Uzupełnij kod metody o operację usunięcia trwałej reprezentacji obiektu za pomocą fasady.

```
@Transactional
public String deleteRequest() {
    Request req =
      (Request) getRequestsDataTable().getRowData();
    ...
    return null;
}
```

22. Przejdź do edycji strony JSF. Dodaj w komponencie `Data Table` kolumnę z przyciskami umożliwiającymi usunięcie żądania wyświetlanego w bieżącym wierszu (jako ostatnią kolumnę tabeli). Użyj w odpowiedni sposób znacznika `<h:commandButton>` i jego własności `value` i `action`.

    Uwaga: W rzeczywistej aplikacji kliknięcie przycisku powinno prowadzić do strony z prośbą o potwierdzenie decyzji o usunięciu żądania.

23. Zapisz wszystkie zmiany. Uruchom aplikację i przetestuj ją usuwając kilka żądań.

24. Sprawdź co się stanie gdy po dodaniu lub usunięciu żądania odświeżysz stronę w przeglądarce (i potwierdzisz ponowne przesłanie danych formularza jeśli przeglądarka o to zapyta).

25. Zaobserwowany w poprzednim kroku ćwiczenia problem jest typowy dla przesyłania danych formularza metodą POST, która w przypadku formularzy JSF jest domyślna. Oczywiście zmiana metody na GET nie byłaby poprawnym rozwiązaniem, gdyż żądania GET nie powinny modyfikować stanu po stronie serwera i ich ponawianie powinno być bezpieczne. Właściwym rozwiązaniem jest zastosowanie wzorca POST-REDIRECT-GET (PRG). Polega on na tym, że aplikacja po obsłużeniu żądania POST instruuje przeglądarkę by ta wysłała żądanie GET pod wskazany adres. Wtedy ewentualne odświeżenie strony przez użytkownika spowoduje powtórne wysłanie tego dodatkowego żądania GET, a nie pierwotnego żądania POST. W JSF aktywacja PRG jest bardzo prosta. Wystarczy do etykiety nawigacyjnej (nazwy strony, do której ma prowadzić nawigacja) dokleić ciąg „?faces-redirect=true". W naszym przypadku etykieta nawigacyjna była pustym łańcuchem znaków, gdyż po dodaniu/usunięciu wpisu użytkownik miał postać na tej samej stronie. W tej sytuacji musimy nazwę strony wskazać jawnie, czyli zwracany przez metody akcji wynik powinien mieć postać „requestsList.xhtml?faces-redirect=true" lub „requestsList?faces-redirect=true". Zmodyfikuj łańcuchy znaków zwracane przez metody do dodawania i usuwania żądań i zaobserwuj czy rozwiązało to problem odświeżania strony przez użytkownika.

Ćwiczenie 2

Celem ćwiczenia jest pokazanie sposobu walidacji danych z wykorzystaniem technologii Bean Validation.

1. Oznacz pole w beanie CDI podpięte pod pole formularza do wprowadzania nowego żądania adnotacją `@Size` standardu Bean Validation. Ustaw minimalną długość tekstu na 3 znaki, a maksymalną na 60 znaków.

2. Na stronie JSF z formularzem dodaj komponent `<h:message>` (obok przycisku do dodawania wpisów) do wyświetlania komunikatu o błędzie walidacji dla pola do wprowadzenia nowego żądania.

3. Przetestuj dodawanie nowych żądań zwracając uwagę na standardowy komunikat o błędzie walidacji.

4. Zastąp standardowy komunikat komunikatem „Request text must be from 3 to 60 characters long." wykorzystując odpowiedni atrybut adnotacji `@Size`.

5. Sprawdź czy nadal poprawnie działa usuwanie żądań.

6. Spraw aby możliwe było usuwanie żądań gdy pole z treścią nowego żądania jest puste.

7. Zawarcie komunikatu o błędzie w atrybucie adnotacji jest złą praktyką, m.in. z punktu widzenia możliwości internacjonalizacji aplikacji. W celu poprawy tego mankamentu wykonaj poniższe kroki:

a) Utwórz w projekcie, w katalogu `src/main/resources` plik o nazwie `ValidationMessages.properties`.

b) Umieść w utworzonym pliku properties poniższy wpis klucz=wartość:

```
request.size=Request text must be from {min} to {max} characters long.
```

c) W adnotacji `@Size` w kodzie klasy komponentu backing bean zastąp treść komunikatem jego kodem w nawiasach klamrowych (`{request.size}`).

d) Jeśli masz czas, możesz przetestować internacjonalizację komunikatu o błędzie walidacji przygotowując drugi, zlokalizowany, plik komunikatów.